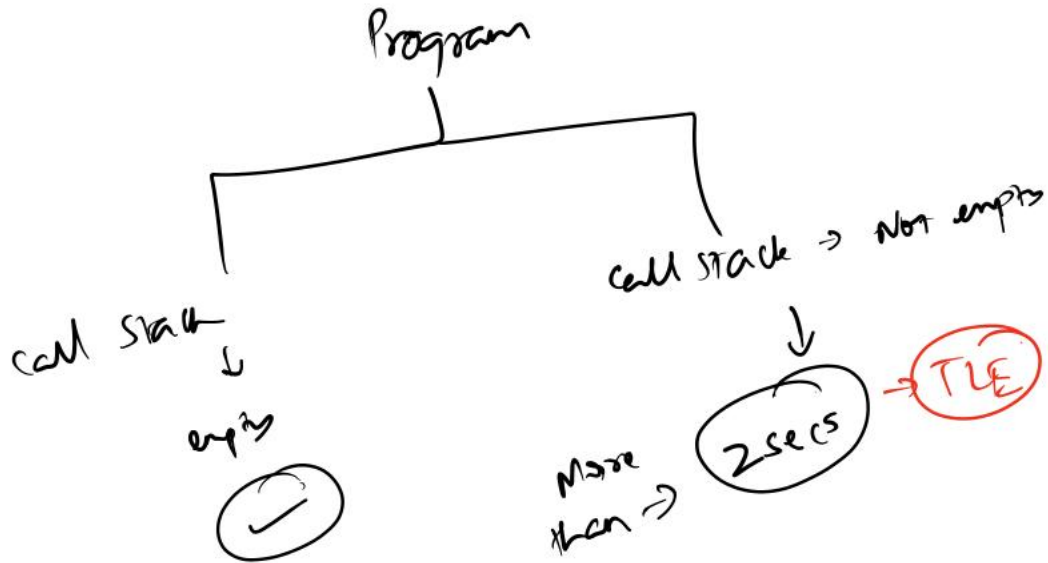
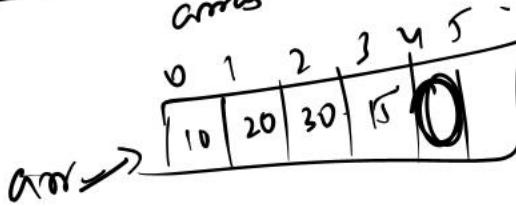


arr →

$x \neq 34$
 $i \mid 0$

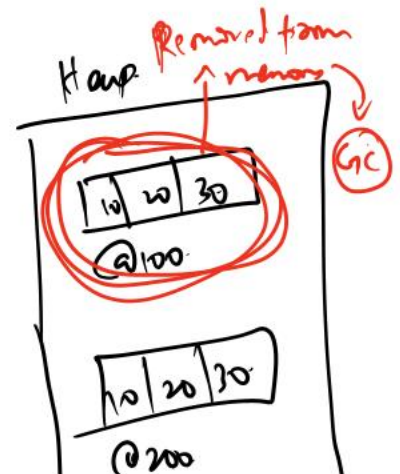
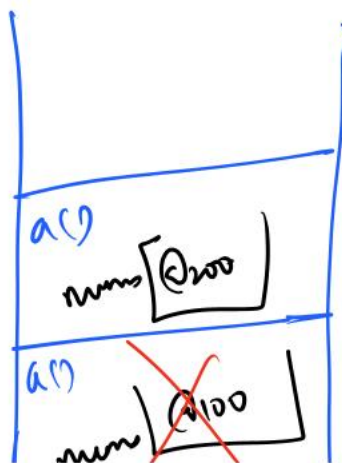
arr →



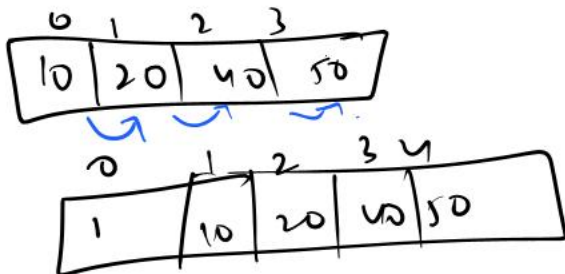
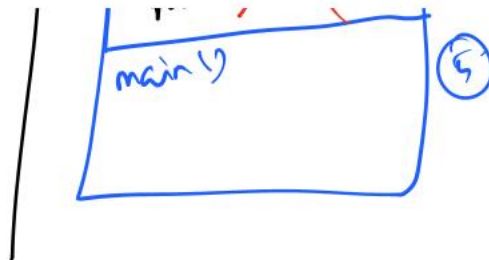
append(10)
 append(20)
 append(30)
 append(15)

def a():

nums = [10, 20, 30]
 print(nums)



5) a()
 6) print("Hello")
 a()



nums.insert(0, 1)

num →

No of shifts $\rightarrow n-1 \approx n \rightarrow O(n)$

nums.insert() $\rightarrow$ nums = []

1. Two Sum Solved ✓

Easy Topics Companies Hint

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution and you may not use the same element twice.

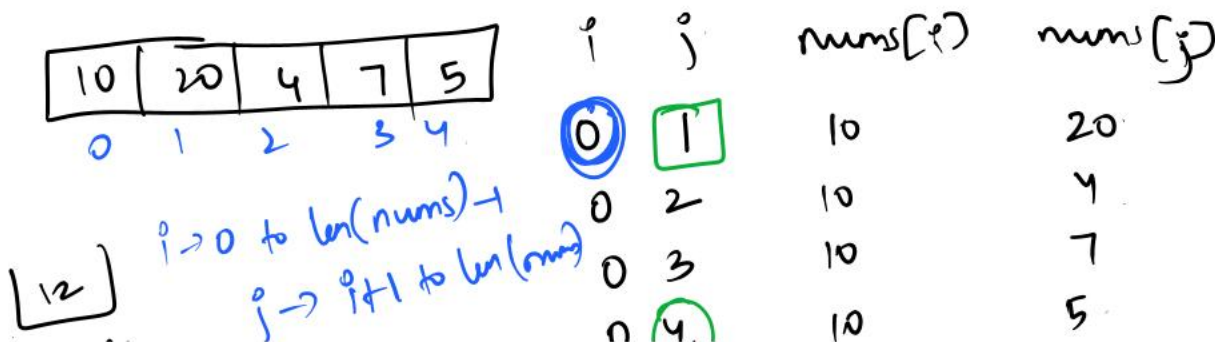
You can return the answer in any order.

Example 1:

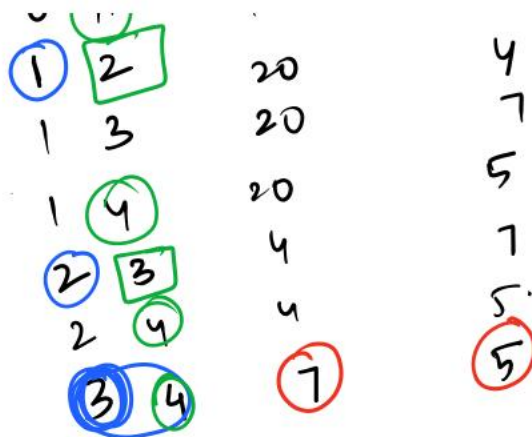
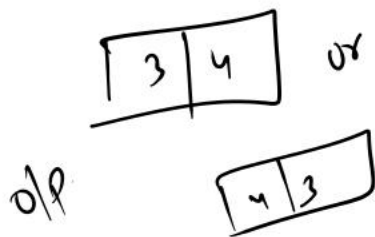
Input: nums = [2,7,11,15], target = 9
Output: [0,1]



Included testcase



target



class Solution:

```
def twoSum(self, nums: List[int], target: int) -> List[int]:
    for i in range(len(nums) - 1):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [j, i]
```

p.No: ① T.C $\rightarrow O(n^2)$ S.C $\rightarrow O(1)$

$n-1 + n-2 + n-3 + n-4 + n-5 + \dots + 2 \Rightarrow n(n-1) \Rightarrow n^2 - n$

217. Contains Duplicate

Solved ✓

Easy Topics Companies

Given an integer array nums, return true if any value appears at least twice in the array, and return false if every element is distinct.

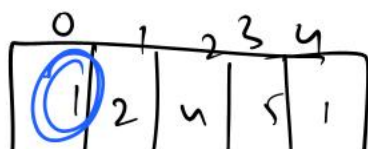
Example 1:

Input: nums = [1, 2, 3, 1]

Output: true



False



class Solution:

```
def containsDuplicate(self, nums: List[int]) -> bool:
    for i in range(len(nums) - 1):  $\rightarrow n$ 
        for j in range(i + 1, len(nums)):  $\rightarrow n$ 
            if nums[i] == nums[j]:
                return True
    return False
```

$$n-1 + n-2 + n-3 + n-4 + n-5 + \dots + 2$$

$$n(n-1) \rightarrow n^2 - n$$

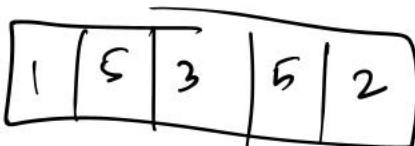
$$O(n^2)$$

1sec $\rightarrow 10^9$ operations

2sec $\rightarrow 10^8$ operations

Compiler $\rightarrow$ run $\rightarrow$ 280ms

$$n^2 \text{ time} \rightarrow (10^5)^2 \rightarrow 10^{10} \rightarrow 2.5 > 2$$



$\rightarrow$



class Solution:

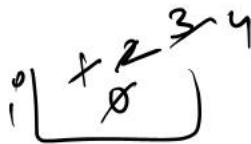
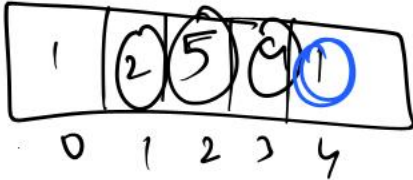
```
def containsDuplicate(self, nums: List[int]) -> bool:
    nums.sort()  $\rightarrow n \log n$ 
    for i in range(len(nums) - 1):  $\rightarrow n$ 
        if nums[i] == nums[i + 1]:
            return True
    return False
```



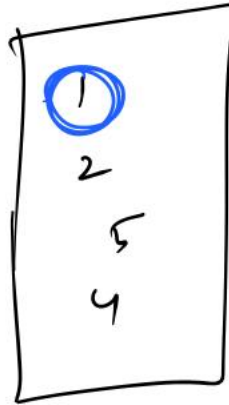
$$\rightarrow \text{Time } O(n \log n)$$

$n + (n \log n)$

Set $\rightarrow$ consist of unique elements $\rightarrow$ No duplicates



Time



$10^5 \rightarrow$ list
 $\downarrow$
unique
 $\downarrow$

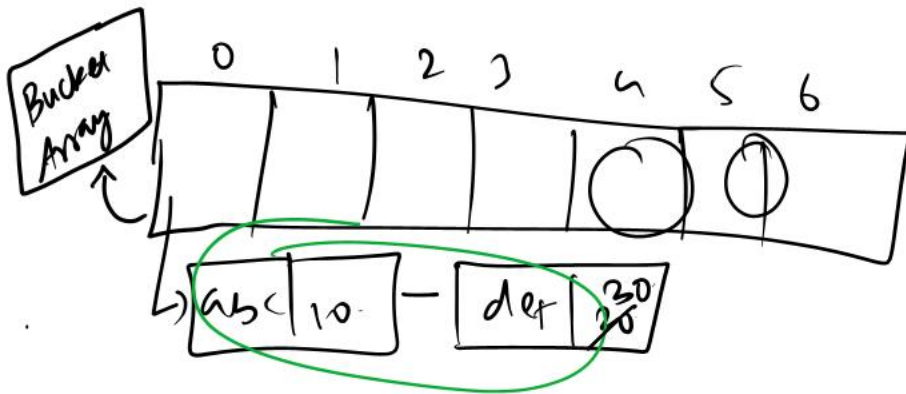
$\ln(\text{set}) \rightarrow 10^5$

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        visited = set()
        for i in range(len(nums)) :  $\rightarrow n$ 
            if nums[i] in visited :
                return True
            else :
                visited.add(nums[i])  $\rightarrow 1$ 
        return False
```

$n \times 1 \rightarrow n$

p.no: 217 T.C $\rightarrow O(n)$ S.C $\rightarrow O(n)$ $\hookrightarrow$ size of set

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return len(nums) != len(set(nums))
```



"abc" : 10

abc → HF → 0

def : 20

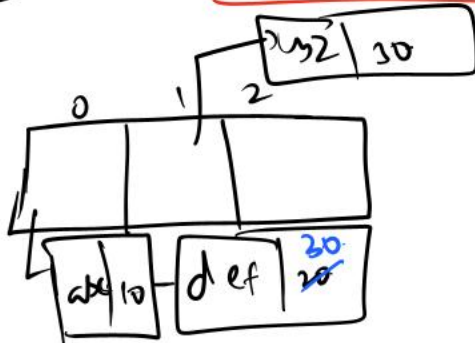
def → HF → 0

```

class Node:
    key
    value
    next
  
```

def : 30

separate chaining



def : 30

abc : 10

xyz : 30

def : 20

"abc" → HF → 0

Q105

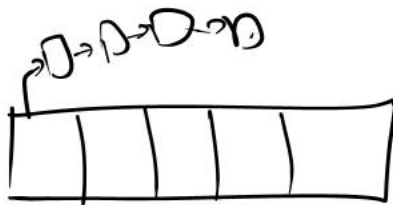
def → HF → 0

xyz → HF → 1

def → HF → 0

```

class Node:
    key
    value
    next
  
```



LOADFACTOR

Size → b.

Rehashing

(10000) → Total

max index → (2000) → (3/10)

2/5 → 0-4

n / n.75

Two sum

$\text{für } \left[\begin{array}{c} 9. \end{array} \right]$
 $\text{oder } \left[\begin{array}{c} 2K \\ 2 \\ 0 \\ 1. \end{array} \right]$

93 2
11/10

```
visited[nums[i]] = i → ①
```

discrimany
572

Solved

1	2	3	2	3
---	---	---	---	---

Easy

Topics

Companies

Hint

Given a **non-empty** array of integers `nums`, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,1]`

Output: 1

1 2 1 4

0 1 2 3 4
5 4 2 5 4

2³ 2² 2¹ 2⁰

5 → 0 1 0 1
4 → 0 0 0 0
—
5 → 0 1 0 1
4 → 0 1 0 0
—
0 0 0 1
2 → 0 0 1 0
—
0 0 1 1
5 → 0 1 0 1
—
0 1 1 0
4 → 0 1 0 0
—
0 0 1 0

2

XOR → same bits cancel

```
class Solution:
```

```
    def singleNumber(self, nums: List[int]) -> int:
```

```
        ans = 0
```

```
        for i in nums:
```

```

    ans ^= i
return ans

```

p-no: 136

T.C $\rightarrow O(n)$

S.C $\rightarrow O(1)$

268. Missing Number

Solved

Easy

Topics

Companies

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return the only number in the range that is missing from the array.

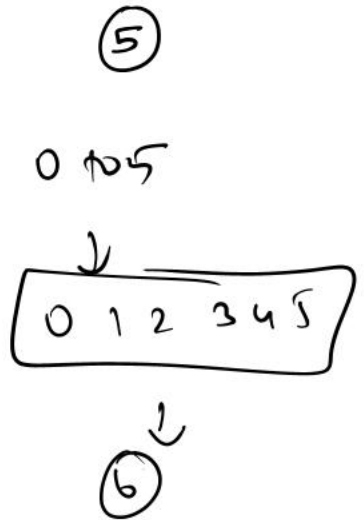
Example 1:

Input: `nums = [3, 0, 1]`

Output: 2

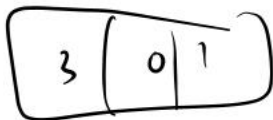
$O(n)$ $\leftarrow$ linear runtime

$O(1)$ $\leftarrow$ constant space



$n(3)$

$$\frac{3 \times (3+1)}{2} = 6$$



9

2 - 101r

```
class Solution:
```

```
    def missingNumber(self, nums: List[int]) -> int:
```

```
        n = len(nums)
```

```
        expectedSum = (n * (n + 1)) // 2
```

```

expectedSum = (n * (n + 1)) // 2
actualSum = sum(nums) → n
return expectedSum - actualSum

```

↓
 p.No : (268) T.C → O(N) s.c → O(1)

7. Reverse Integer

Solved ✓

Medium

Topics

Companies

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$ then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$

Output: 321

-123

↓

-321

$x \setminus \begin{matrix} 12 \\ 3 \end{matrix} \rightarrow 0$

321
 32
 3
 0
 num

digit
 7
 2
 3

$digit = x \% 10$
 $num = (num \times 10) + digit$
 $x // = 10$

$(x > 0) \leftarrow \text{loop}$

```

class Solution:
    def reverse(self, x: int) -> int:
        sign = 1
        if x < 0 :
            sign = -1
        num = 0
        x = sign * x
        while x > 0 :
            digit = x % 10
            num = (num * 10) + digit
            x //= 10
        if sign == -1 :
            num *= sign
        if (num < -(2 ** 31)) or (num > ((2 ** 31) - 1)):
            return 0
        return num

```

→ P.No ⑦

→ T.C → $O(k)$

→ S.C → $O(1)$

$k \rightarrow$ no. of digits
in n